

CS222 Assignment 3: เรียนรู้ Process

Out: 12 มีค 69

390 คะแนน

Due: 26 มีค 69

Assignment นี้เป็นงานเดี่ยว มีวัตถุประสงค์ให้ นศ เรียนรู้การเขียนโปรแกรมเพื่อสร้าง Process และศึกษา Process บนระบบ Linux

นศ สามารถใช้ระบบ Linux จากที่ต่อไปนี้

1. ห้อง Lab ของสาขาวิชา
2. เครื่องคอมพิวเตอร์ของ นศ เอง โดยใช้ระบบ Window Subsystem for Linux 2
 - a. WSL2 บน Windows 11
 - i. <https://www.youtube.com/watch?v=eld6K8d0v6o>
 - b. WSL2 บน Windows 10
 - i. https://www.youtube.com/watch?v=_fntjriRe48&list=PLhfrWillOoKNMHhB39bh3XBpoLxV3f0V9
3. เครื่องคอมพิวเตอร์ของ นศ เอง โดย
 - a. ติดตั้ง Virtualbox <https://www.youtube.com/watch?v=VSar76Y7wPs> แล้วจึง
 - b. ติดตั้ง Ubuntu 24.04 Linux <https://www.youtube.com/watch?v=QXdFTEPXJ4M>
4. รันด้วยวิธีการอื่น เช่นระบบ AWS Cloud โดยหาข้อมูลจาก Internet
5. หาก นศ ไม่มีคอมพิวเตอร์หรือคอมพิวเตอร์มีปัญหา ขอให้แจ้งอาจารย์

ใน Lab Assignment ของวิชา CS222 เรามีนโยบายเกี่ยวกับการทำ Assignment ดังนี้

1. วิชานี้สนับสนุนให้ (1) นศ ทำงานด้วยตนเอง และ (2) ให้ นศ ปรึกษาหารือกันเพื่อตรวจสอบความถูกต้องหรือแก้ปัญหาร่วมกัน ภายในกลุ่มหรือข้ามกลุ่มก็ได้ แต่ไม่อนุญาตให้ copy รายงานหรือคำตอบ หลังจากที นศ หารือกันแล้วร่วมกันแล้ว ให้แต่ละคนเขียนคำตอบ เขียน code และเขียนรายงานด้วยคำพูดของตนเอง
ในกรณีที่มีการ Copy รายงาน จะได้คะแนน Assignment จะเป็น 0 ทั้งผู้ copy และผู้ให้ copy
Soft Skills: ฝึกการสร้างเชื่อมั่นในตนเองและความมีศักดิ์ศรี และไม่ช่วยผู้อื่นในทางที่ผิด
2. นศ สามารถใช้ AI เพื่อช่วยหาคำตอบและเพิ่มความเข้าใจได้ แต่ไม่สนับสนุนให้ copy หรือ quote คำตอบจาก AI มาในรายงานโดยตรง หลังจากได้คำตอบ ไม่ควรเชื่อ AI (หรือเชื่ออาจารย์) ทันที ให้พิจารณาความถูกต้อง แล้ว เขียน code ด้วยตนเอง และตอบคำถามด้วยคำพูดของตนเอง
3. ในการส่งงานเข้าสู่ระบบ MS Team ขอให้ นศ เขียนชื่อ และรหัส นศ ให้ชัดเจน และ upload รายงานเป็น pdf ใน MS Team Assignment
4. ในกรณีส่งงาน Late จะมีการหักคะแนนวันละ 10 เปอร์เซนต์

ข้อ 1: จาก Slide 06 จงศึกษาเรื่องการพัฒนาโปรแกรมภาษา C ตั้งแต่หน้า 24 ถึง 29 และเขียนโปรแกรม `hellomain.c` `hellosub1.c` `hellosub2.c` และคอมไพล์และ link object files เพื่อสร้างโปรแกรม hello ในการส่งงานให้ นศ แสดง source code พร้อมทั้งผลการรันโปรแกรมในรายงาน (10 คะแนน)

ข้อ 2: ให้ นศ ค้นหาข้อมูลเกี่ยวกับ system call (หรือกลุ่มของ system call) ต่อไปนี้จาก internet และอธิบายอย่างสั้นๆว่า (1) system call เหล่านี้ทำหน้าที่อะไร (2) ในการใช้งานต้องอ้างอิง include file อะไร (40 คะแนน)

a. fork	b. wait	c. waitpid	d. execlp
e. exit	f. getppid	g. getpid	h. execv

ข้อ 3: จาก Slide 06 จง คำนวณด้วยตนเองและตอบอย่างสั้นว่า `exec*()` system call มีหน้าที่ทำอะไร และ `exec` system calls ต่อไปนี้ มีความแตกต่างกันอย่างไร (40 คะแนน)

Function	pathname	filename	fd	Arg list	argv[]	environ	envp[]
execl	•			•		•	
execlp		•		•		•	
execle	•			•			•
execv	•				•	•	
execvp		•			•	•	
execve	•				•		•
fexecve			•		•		•
(letter in name)		p	f	l	v		e

```
[ubuntu@vtest:~/CS222$ cat x
#!/bin/bash
r=$(( ${1} - 1 ))
echo $r
echo PID=$$
n="./x"
$ n $r
ubuntu@vtest:~/CS222$
```

ภาพ 1(a)

```
[ubuntu@vtest:~/CS222$ cat y
r=$(( ${1} - 1 ))
echo $r
echo PID=$$
n="./z"
exec $n $r
ubuntu@vtest:~/CS222$
```

ภาพ 1(b)

ข้อ 4: ให้เขียน shell script x ดัง ภาพ 1(a) และ script y ดังภาพ 1(b) ให้รัน script และกด Ctrl + C เพื่อหยุดการทำงานของ script แล้วตอบคำถามต่อไปนี้ (15 คะแนน)

1. จงอธิบายแต่ละคำสั่งใน script x และ y
2. จง capture หน้าจอผลการรัน `$ ./x 200` และอธิบายว่าทำไมค่า r จึงลดลงและ เพราะเหตุใดค่า PID ในการรันแต่ละครั้งจึงเปลี่ยนไป
3. จง capture หน้าจอผลการรัน `$ ./y 200` และอธิบายว่าผลลัพธ์ของ y ต่างจากการรัน x อย่างไร เพราะเหตุใด

```
[ubuntu@vtest:~/CS222$ cat a
#!/bin/bash
m=$(( ${m} - 1 ))
echo $m
echo PID=$$
${0}
```

ภาพ 2(a)

```
[ubuntu@vtest:~/CS222$ cat b
#!/bin/bash
m=$(( ${m} - 1 ))
echo $m
echo PID=$$
exec ${0}
```

ภาพ 2(b)

ข้อ 5: ให้เขียน shell script a ดัง ภาพ 2(a) และ script b ดังภาพ 2(b) ให้รัน script และกด Ctrl + C เพื่อหยุดการทำงานของ script แล้วตอบคำถามต่อไปนี้ (25 คะแนน)

1. จงอธิบายแต่ละคำสั่งใน script a และ b
2. จงตอบว่าจะต้องรันคำสั่งอะไร ก่อนรันคำสั่ง

`$ ./a`

จึงจะทำให้ผลลัพธ์ (ไม่รวม ค่า PID ที่ไม่ต้องเหมือนกัน แต่ค่า PID สามารถเพิ่มขึ้นได้เหมือนของ `./x`) เหมือนรัน และเพราะเหตุใดจึงเป็นเช่นนั้น

`$ ./x 200`

3. จง capture หน้าจอผลการรัน `$ ./a` และอธิบายว่าทำไมค่า m จึงลดลงและ เพราะเหตุใดค่า PID ในการรันแต่ละครั้งจึงเปลี่ยนไป
4. จง capture หน้าจอผลการรัน `$ ./b` อธิบายว่าผลลัพธ์ต่างจากการรัน `./a` อย่างไร เพราะเหตุใด
5. ถ้าต้องการให้การประมวลผลของ `./b` หยุดที่ค่า `m = -200` และพิมพ์คำว่า “done” ออกสู่หน้าจอจะต้องเปลี่ยน script b อย่างไร อธิบายคำสั่งที่ใช้และ capture ผลการรัน

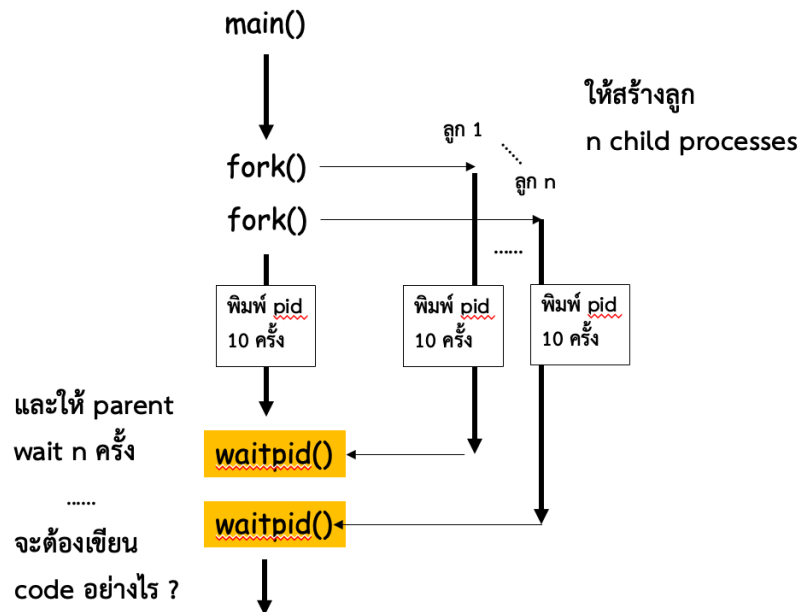
ข้อ 6: จาก Slide 06 จงเขียนและรันตัวอย่างโปรแกรมต่อไปนี้ และอธิบายสั้นๆว่าโปรแกรมเขียนขึ้นเพื่อแสดงการประมวลผลอะไร (140 คะแนน)

โปรแกรม	หน้า	คำสั่งเพิ่มเติม
ex1.x	36	
ex2.c	39	
ex2-gen.c	46	ให้ นศ Visualize โปรแกรม ex02-gen.c เมื่อ $n = 4$
ex3.c	50	
ex4.c	52	อธิบายผลของคำสั่ง ps -fu และ ps -ef ว่าเกิด orphan อย่างไร
ex5.c	56	อธิบายผลของคำสั่ง ps -fu และ ps -ef ว่าเกิด zombie อย่างไร
ex5-1.c และ ex5-2.c	58 และ 61	อธิบายผลการรันในแง่ Status ของ Process ว่า Parent มีสถานะต่างกันอย่างไร เพราะเหตุใด
ex6.c	63	
ex7.c	65	
ex8.c, ex9.c, ex10.c	79, 81, 85	
shell1.c	87	

ข้อ 7: จากโปรแกรม ex7.c ให้ขยายโปรแกรมให้สร้าง โพรเซสลูกจำนวน n โพรเซส โดยรับค่า n เป็นค่า Parameter ตอนเรียกโปรแกรม สมมติว่าแม่ไฟล์โปรแกรมเป็น ex7 executable เมื่อเรียก

\$./ex7 8

หมายถึงค่า $n = 8$ โดยที่โพรเซสลูกทั้งหมดพร้อมทั้ง parent โพรเซสจะต้อง พิมพ์ค่า Process ID ของตนเอง ออกสู่หน้าจอเป็นจำนวน 10 ครั้ง ไปพร้อมๆกัน แล้วหลังจากนั้นให้ Parent Process ใช้ waitpid system call รอให้ลูกทั้งหมดจบการทำงาน ดังภาพที่ 3 ให้แสดง source code และ capture ผลลัพธ์ใส่ในรายงาน (40 คะแนน)



ภาพที่ 3

ข้อ 8: จงเขียนโปรแกรม ให้สร้างโปรเซสลูกซ้อนกันดังภาพที่ 4 จำนวน n ชั้น (โดยรับค่า n จาก Command Line Parameter เหมือนในข้อก่อนหน้า) นศ สามารถใช้วิธีการ recursive หรือวิธีการ loop ก็ได้ เลือกวิธีใดวิธีหนึ่ง (หลังจากเลือกแล้ว นศ อาจลองพิจารณาด้วยว่าถ้าจะทำวิธีที่ไม่ได้เลือกจะต้องเขียนโปรแกรมอย่างไร) มีข้อกำหนดดังนี้ ให้แสดง source code และ capture ผลลัพธ์ใส่ในรายงาน (40 คะแนน)

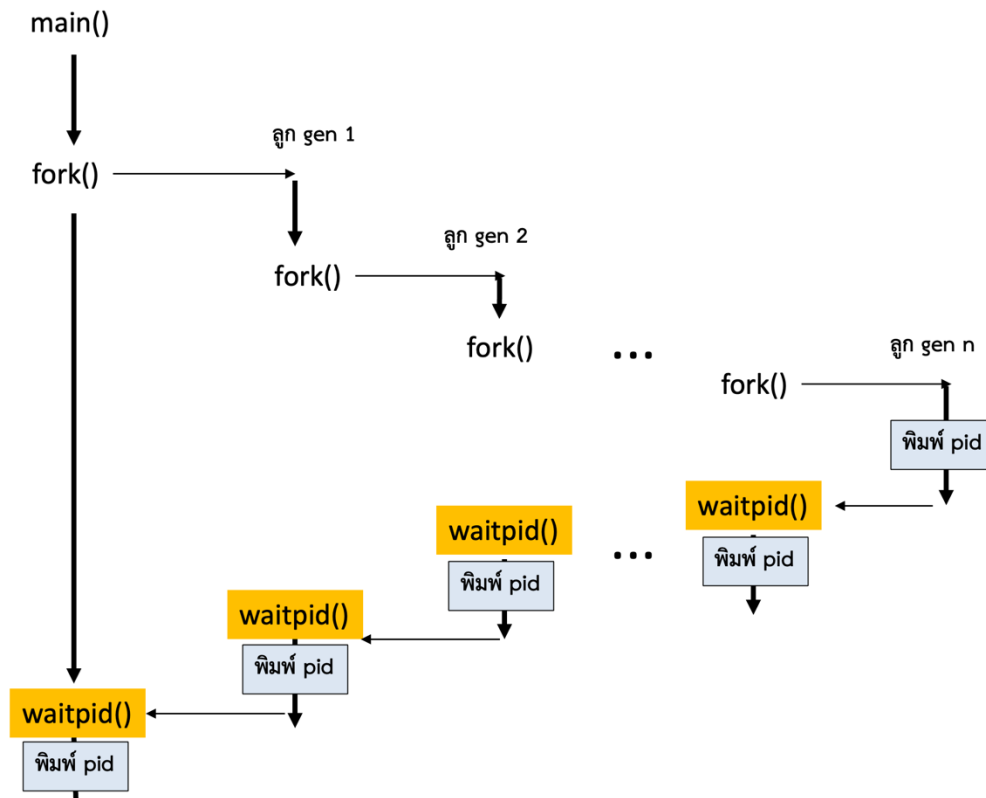
- สร้างโปรเซสลูก เรียกว่าโปรเซส Gen 1 แล้วให้ main รอให้ลูกจบการประมวลผล แล้ว
- ให้โปรเซสลูก fork สร้างโปรเซสหลาน เรียกว่าโปรเซส Gen 2 แล้วให้โปรเซส Gen 1 รอให้ Gen2 จบการประมวลผล และ
- ให้โปรเซส Gen 2 fork โปรเซส Gen 3 แล้วให้โปรเซส Gen 2 รอให้ Gen 3 จบการประมวลผล และ
- ให้โปรเซส Gen 3 fork โปรเซส Gen 4 แล้วให้โปรเซส Gen 3 รอให้ Gen 4 จบการประมวลผล และ
- ทำเช่นนี้ไปเรื่อยๆ ให้โปรเซส Gen N fork โปรเซส Gen N+1 แล้วให้โปรเซส Gen N รอให้ Gen N+1 จบการประมวลผล และ
- ให้แต่ละโปรเซสพิมพ์ค่า pid ของตนเอง (อาจใช้ getpid())

“My PID is <PID>\n”

และค่า pid ของโปรเซส parent (อาจใช้ getppid())

“My PID is <Parent PID>\n”

ให้ นศ พิมพ์โค้ดของโปรแกรมใส่ในรายงานพร้อมทั้งเขียนคอมเมนต์อธิบายคร่าวๆ และให้ capture หน้าจอผลการประมวลผลและอธิบายจากผลการรันว่าทำไมโปรแกรมถึงทำงานได้ถูกต้องตามที่ต้องการ



ภาพที่ 4

ข้อ 9: จากโปรแกรม shell1.c ใน Slide ขอให้ขยายโปรแกรมเป็น shell2.c โดยมีข้อกำหนดดังนี้ (40 คะแนน)

1. ห้ามใช้ stdlib function system()

NAME

system – pass a command to the shell

LIBRARY

Standard C Library (libc, -lc)

SYNOPSIS

#include <stdlib.h>

int

system(const char *command);

2. โปรแกรม shell2.c ต้องสามารถรับค่า parameters ได้ (Hint: ใช้ strtok() หรือวิธีการตัดคำอย่างอื่น) ตัวอย่างคำสั่งเช่น

\$ ls -l

\$ df -h

เป็นต้น

3. ถ้าผู้ใช้ป้อน string “exit” ให้ shell2 จบการประมวลผล

4. ถ้าผู้ใช้ป้อน string “pwd” ให้แสดงค่า current directory (Hint: ใช้ getcwd() system call)